Cairo 컴파일러 프로젝트 구조 분석

개요

Cairo 컴파일러는 Rust로 구현된 대규모 워크스페이스 프로젝트로, 모듈화된 아키텍처를 통해 Cairo 언어의 파싱부터 CASM 코드 생성까지의 전체 컴파일 파이프라인을 구현합니다. 이 문서는 프로젝트의 구조, 각 컴포넌트의 역할, 의존성 관계, 그리고 빌드 시스템에 대해 상세히 분석합니다.

Cargo 워크스페이스 구성

워크스페이스 설정

Cairo 컴파일러는 Cargo 워크스페이스를 사용하여 40개 이상의 크레이트를 통합 관리합니다:

```
[workspace]

resolver = "2"

members = [

# 핵심 라이브러리 크레이트들

"crates/cairo-lang-casm",

"crates/cairo-lang-compiler",

"crates/cairo-lang-parser",

"crates/cairo-lang-semantic",

"crates/cairo-lang-sierra",

# ... 기타 30+ 크레이트들

# 실행 가능한 바이너리들
```

```
"crates/bin/cairo-compile",
"crates/bin/cairo-run",
"crates/bin/starknet-compile",
# ... 기타 바이너리들
]
```

버전 관리 전략

모든 크레이트는 통일된 버전 관리를 사용합니다:

- **현재 버전**: 2.12.0
- **Rust 에디션**: 2024
- **최소 Rust 버전**: 1.86
- **라이선스**: Apache-2.0

핵심 크레이트 분류 및 역할

1. 컴파일러 프론트엔드

cairo-lang-parser

역할: Cairo 소스 코드의 구문 분석

- **주요 기능**:
- 토큰화 (Lexical Analysis)
- 구문 분석 (Syntax Analysis)
- AST (Abstract Syntax Tree) 생성
- **핵심 의존성**:

- `cairo-lang-syntax` : 구문 정의
- `cairo-lang-primitive-token` : 기본 토큰 정의
- `unescaper` : 문자열 이스케이프 처리

cairo-lang-syntax

역할: Cairo 언어의 구문 정의 및 AST 구조

- **특징**: 코드 생성을 통한 구문 정의 자동화
- **관련 크레이트**: `cairo-lang-syntax-codegen`

cairo-lang-semantic

역할: 의미 분석 및 타입 체킹

- **주요 기능**:
- 타입 추론 및 검증
- 스코프 해결
- 트레이트 해결
- 제네릭 인스턴스화
- **핵심 의존성**:
- `cairo-lang-defs` : 정의 관리
- `cairo-lang-plugins` : 플러그인 시스템

2. 중간 표현 및 변환

cairo-lang-lowering

역할: 고수준 표현을 중간 표현으로 변환

- **변환 과정**: HIR (High-level IR) → MIR (Mid-level IR)

- **주요 기능**:
- 패턴 매칭 해제
- 제어 흐름 단순화
- 함수 인라이닝

cairo-lang-sierra

역할: Sierra 중간 표현 정의 및 처리

- **특징**:
- LALRPOP를 사용한 파서 생성
- 가스 효율적인 중간 표현
- 증명 생성에 최적화된 구조

cairo-lang-sierra-generator

역할: Lowered 표현에서 Sierra 코드 생성

- **기능**: MIR → Sierra IR 변환

3. Sierra 최적화 모듈들

cairo-lang-sierra-gas

역할: Sierra 코드의 가스 계산 및 최적화

- **기능**: 각 Sierra 명령어의 가스 비용 분석

cairo-lang-sierra-ap-change

역할: AP (Allocation Pointer) 변화 추적

- **기능**: 메모리 할당 최적화를 위한 AP 변화 분석

cairo-lang-sierra-type-size

역할: Sierra 타입의 크기 분석

- **기능**: 메모리 레이아웃 최적화

4. 코드 생성 백엔드

cairo-lang-casm

역할: CASM (Cairo Assembly) 정의 및 처리

- **기능**: Cairo VM에서 실행 가능한 어셈블리 코드 표현

cairo-lang-sierra-to-casm

역할: Sierra에서 CASM으로의 최종 변환

- **주요 기능**:
- Sierra 명령어의 CASM 매핑
- 레지스터 할당
- 메모리 레이아웃 최적화

5. 실행 및 유틸리티

cairo-lang-runner

역할: Cairo 프로그램 실행 환경

- **기능**:
- Cairo VM 인터페이스
- 프로그램 실행 관리
- 증명 생성 지원

cairo-lang-compiler

역할: 전체 컴파일 파이프라인 조정

- **기능**: 모든 컴파일 단계를 통합하는 메인 컴파일러

6. Starknet 특화 모듈들

cairo-lang-starknet

역할: Starknet 스마트 컨트랙트 컴파일 지원

- **기능**:
- 컨트랙트 특화 구문 처리
- 스토리지 변수 처리
- 이벤트 시스템

cairo-lang-starknet-classes

역할: Starknet 컨트랙트 클래스 생성

- **기능**:
- 컨트랙트 클래스 해시 계산
- ABI 생성
- 배포 아티팩트 생성

7. 개발 도구 및 유틸리티

cairo-lang-formatter

역할: Cairo 코드 포매팅

- **기능**: 일관된 코드 스타일 적용

cairo-lang-test-plugin & cairo-lang-test-runner

역할: 테스트 시스템

- **기능**:

- 단위 테스트 실행
- 통합 테스트 지원

cairo-lang-doc

역할: 문서 생성

- **기능**: 코드에서 문서 자동 생성

실행 가능한 바이너리들

주요 컴파일러 도구들

cairo-compile

목적: Cairo 소스 코드를 Sierra 또는 CASM으로 컴파일

```
cairo-compile  --single-file  program.cairo  --output  program.sierra
```

cairo-run

목적: Cairo 프로그램 직접 실행

```
cairo-run  --single-file  program.cairo
```

starknet-compile

목적: Starknet 컨트랙트 컴파일

```
starknet-compile  contract.cairo  contract.json
```

cairo-format

목적: Cairo 코드 포매팅

```
cairo-format  --recursive  src/
```

개발 및 분석 도구들

sierra-compile

목적: Sierra 코드를 CASM으로 컴파일

cairo-size-profiler

목적: 컴파일된 코드의 크기 분석

get-lowering

목적: Lowering 과정의 중간 결과 출력

의존성 그래프 분석

계층적 의존성 구조

```
cairo-lang-compiler (최상위)
├── cairo-lang-sierra-generator
| ├── cairo-lang-lowering
| | ├── cairo-lang-semantic
| | | ├── cairo-lang-defs
| | | | └── cairo-lang-parser
| | | | └── cairo-lang-syntax
```

```
│ │ │ └── cairo-lang-plugins
│ │ └── cairo-lang-debug
│ └── cairo-lang-sierra
└── cairo-lang-sierra-to-casm
├── cairo-lang-casm
├── cairo-lang-sierra-gas
├── cairo-lang-sierra-ap-change
└── cairo-lang-sierra-type-size
```

공통 유틸리티 의존성

모든 크레이트가 공통으로 의존하는 기반 모듈들:

cairo-lang-utils

역할: 공통 유틸리티 함수 및 매크로

- **기능**:
- 컬렉션 유틸리티
- 에러 처리 헬퍼
- 직렬화 지원

cairo-lang-filesystem

역할: 파일 시스템 추상화

- **기능**:
- 가상 파일 시스템
- 파일 ID 관리

- 소스 맵핑

cairo-lang-diagnostics

역할: 진단 및 에러 보고 시스템

- **기능**:
- 에러 메시지 포매팅
- 진단 정보 수집
- IDE 통합 지원

외부 의존성 분석

핵심 외부 라이브러리들

Salsa (0.23.0)

- **용도**: 증분 컴파일을 위한 쿼리 시스템
- **역할**: 컴파일러의 캐싱 및 의존성 추적

LALRPOP (0.22.2)

- **용도**: 파서 생성기
- **사용처**: Sierra 파서 생성

Rayon (1.10.0)

- **용도**: 병렬 처리
- **역할**: 컴파일 과정의 병렬화

Cairo VM (2.4.0)

- **용도**: Cairo 프로그램 실행
- **역할**: 실제 프로그램 실행 및 증명 생성

빌드 시스템 및 컴파일 옵션

빌드 프로파일 구성

Release 프로파일

```
[profile.release]
overflow-checks = true # 정수 오버플로우 검사 활성화
```

CI 개발 프로파일

```
[profile.ci-dev]
inherits = "dev"
incremental = false # 증분 컴파일 비활성화

# 외부 의존성 최적화
[profile.ci-dev.package."*"]
opt-level = 1

# 특정 크레이트 고도 최적화
[profile.ci-dev.package."salsa"]
opt-level = 3
```

코드 품질 도구

Clippy 설정

```
# clippy.toml
enum-variant-name-threshold = 1
```

Rustfmt 설정

```
# rustfmt.toml
edition = "2024"
max_width = 100
imports_granularity = "Module"
group_imports = "StdExternalCrate"
```

빌드 스크립트들

Cairo 포매팅

```
# scripts/cairo_fmt.sh
cargo run --profile=ci-dev --bin cairo-format -- --recursive "$@"
```

테스트 실행

```
# scripts/cairo_test.sh
cargo run --profile=ci-dev --bin cairo-test -- "$@"
```

Corelib 구조

Cairo 표준 라이브러리

Cairo의 핵심 라이브러리는 별도의 Scarb 프로젝트로 관리됩니다:

```
[package]
name = "core"
version = "2.12.0"
edition = "2024_07"
no-core = true # 자기 참조 방지
```

특징:

- **실험적 기능**: coupons, negative_impls, associated_item_constraints
- **자체 완결성**: 다른 core 라이브러리에 의존하지 않음
- **Cairo 네이티브**: Cairo 언어로 직접 구현

프로젝트 구조의 설계 원칙

1. 모듈화 및 관심사 분리

- 각 컴파일 단계별로 독립적인 크레이트 구성
- 명확한 인터페이스를 통한 단계 간 통신
- 테스트 가능한 작은 단위로 분할

2. 증분 컴파일 지원

- Salsa 기반의 쿼리 시스템 활용
- 변경된 부분만 재컴파일하는 효율적인 구조

- 캐싱을 통한 컴파일 속도 최적화

3. 확장성 및 플러그인 지원

- 플러그인 시스템을 통한 기능 확장
- 매크로 시스템 지원
- 커스텀 속성 및 코드 생성 기능

4. 다중 타겟 지원

- 일반 Cairo 프로그램과 Starknet 컨트랙트 모두 지원
- 타겟별 특화된 컴파일 경로 제공
- 공통 기반 위에 특화 기능 구축

결론

Cairo 컴파일러의 프로젝트 구조는 복잡한 컴파일 파이프라인을 효과적으로 관리하기 위해 고도로 모듈화된 아키텍처를 채택하고 있습니다. 각 크레이트는 명확한 책임을 가지며, 의존성 그래프는 컴파일 과정의 자연스러운 흐름을 반영합니다. 이러한 구조는 유지보수성, 테스트 가능성, 그리고 확장성을 모두 고려한 설계로 평가됩니다.